

Chapter 4 — Software Design and Implementation

CGRA-Accelerated MNIST CNN Inference on Zynq-7000

Engineering Graduation Thesis

May 12, 2026

Context. The chapter that follows documents the software stack of a silicon-validated bare-metal demo running on a PYNQ-Z2 development board (Xilinx Zynq XC7Z020, Cortex-A9 @ 666 MHz, custom CGRA @ 50 MHz). The locked-demo head commit is tagged `baremetal-mnist-fc-demo-locked`; all code references and register addresses in this chapter were verified against that commit. Measured silicon results on 201 frames: **CGRA-FC 1.50 ms (668 FPS)**, **ARM-INT-FC 5.60 ms (179 FPS)**, **ARM-VFP-FC 4.36 ms (230 FPS)**; $3.74\times$ wall-clock speedup of CGRA-FC over scalar-INT ARM at 97% classification accuracy.

Contents

4.1 Design Requirements	1
4.2 Software Design	2
4.2.1 System Software Architecture	2
4.2.2 Architectural Partitioning	4
4.2.3 Per-Frame Execution Flowchart	5
4.3 Software Implementation	9
4.3.1 Initialization of Memory Regions	9
4.3.2 Initialization of Weights and CGRA Configuration	10
4.3.3 Reading Input Data	12
4.3.4 Preprocessing of Input Data	12
4.3.5 Hardware Control: APB and DMA Subsystems	13
4.3.6 Output Layers and HDMI Display	15

4.1 Design Requirements

The software stack supporting the CGRA-accelerated MNIST inference demo must satisfy three orthogonal classes of requirements: **data-path correctness**, **control-path determinism**, and **subsystem coordination**. The data path orchestrates the migration of 28×28 input images, 105 KB of pre-quantized weight blobs, and

per-layer activations between three storage tiers — DDR3 main memory (0x00200000–0x002FFFFFF), the CGRA tile memory (4 banks $\times$ 4 KiB SRAM, accessed through the 0x10000000 window), and the per-PE scratchpad (1024 words $\times$ 32 bits per Processing Element, accessed through the 0x40000000 window). The software must marshal each layer’s input tensor into the correct memory region, configure the integrated DMA controller with the appropriate source/destination/length triple, and recover the layer’s output activations through a memory-mapped result register file without violating the AXI4 burst contract or the CGRA accumulator’s 40-bit saturation envelope.

The control path must, simultaneously, configure the CGRA’s 16 Processing Elements (PEs) into a context-multiplexed dense-matrix-multiply kernel and trigger the Control Unit (CU) to execute that kernel deterministically — without operating-system jitter, cache-line interference, or interrupt latency that would corrupt the cycle-counter measurements forming the thesis’ performance claims. To meet this constraint, the implementation is hosted on a bare-metal Cortex-A9 ELF rather than under Linux: the host code runs from on-chip memory (OCM), drives the CGRA through its 30-register APB control-status register (CSR) file, and renders results to an HDMI framebuffer in DDR through a direct memory-mapped IO pattern, with no operating system in the critical loop. This design preserves a sub-1% jitter envelope on the per-frame cycle measurements while still supporting a real-time visual demonstration to a defense audience.

4.2 Software Design

4.2.1 System Software Architecture

Before describing the per-frame execution flow, this subsection establishes the high-level layering of the software stack and the hardware blocks it drives. Figure 4.1 gives a block-diagram view of the complete system: the application code on the left talks to the CGRA accelerator on the right through two distinct interfaces — a narrow control plane over APB and a wide data plane over AXI4 — and both sides share a single DDR3 memory through which all bulk operands pass.

Three layers on the PS side. The host stack is a strict three-layer cake. The *bare-metal runtime* at the bottom — the reset vector in `start.s`, the GIC driver in `gic.h,c`, and the polled UART driver in `uart.h` — runs with the Memory Management Unit disabled, the L1 data cache off, and no operating system whatsoever; it provides exactly enough scaffolding for C code to execute from on-chip memory and for hardware peripherals to be reachable at their physical addresses. Above this, a thin *driver / hardware-abstraction layer* exposes the CGRA’s register-level operations as inline C functions (`cgra.h`), the silicon-validated FC kernel chains as higher-level routines (`cgra_kernels_cnn.h`), and the HDMI framebuffer and rendering primitives (`hdmi_bm.h`, `fb_lib_bm.h`). At the top, the *application layer* — the demo’s `main()`, the VFP convolution reference, the FC post-processing, and the argmax classifier — combines those abstractions into the per-frame inference loop. This layering keeps the application code free of any direct register pokes and keeps the register-level code free of any application-specific concepts.

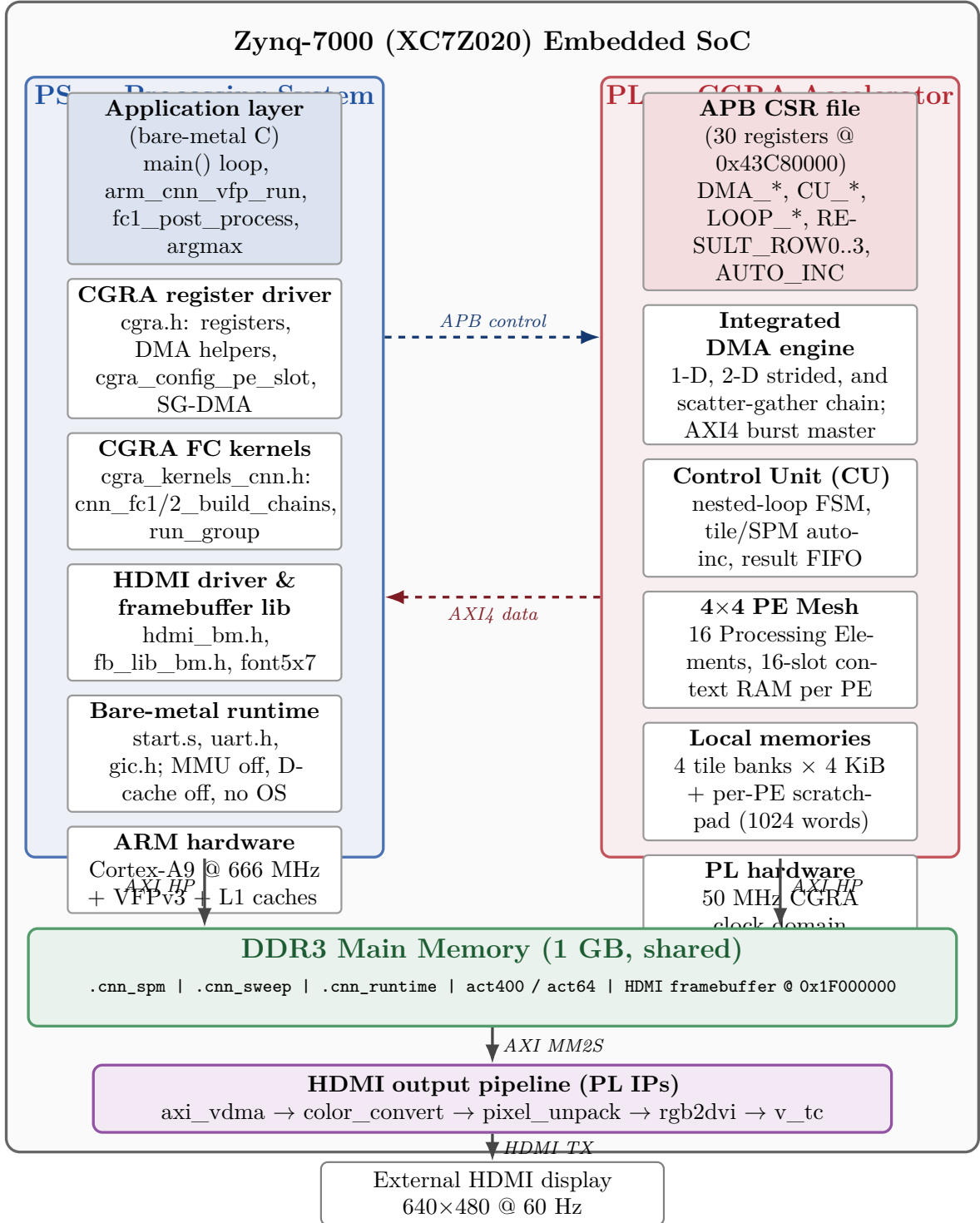


Figure 4.1: System software architecture and ARM-CGRA interaction. The blue (PS) and red (PL) blocks are physical halves of the Zynq SoC. Inside the PS block, sub-blocks are layered from the application code (top) down to the bare-metal runtime and the underlying ARM hardware. Inside the PL block, the CGRA’s externally-visible interfaces (APB CSR and DMA) are at the top, with the compute fabric beneath. Dashed arrows between PS and PL are the two logical interfaces: APB carries narrow control words; AXI4 carries wide bulk data. Both halves share the DDR3 memory below them; the HDMI output pipeline draws its framebuffer from the same DDR.

The CGRA’s two external interfaces. Seen from the host, the CGRA exposes exactly two interfaces. The *APB control-status interface* (Advanced Peripheral Bus, 32 bits, serialized) is a narrow channel used by the host to write configuration words to the CGRA’s 30 registers and read back status flags. Every operation the host initiates — start a DMA, start the CU, change the hardware-loop count, drain the result registers — is one or more APB writes or reads. The *AXI4 master interface* (Advanced eXtensible Interface, 64-bit data, burst-capable) is the wide data channel by which the CGRA’s integrated DMA engine reads operands from DDR and writes them into the tile memory, the per-PE scratchpad, or the PE configuration RAM, and through which it can also write results back to DDR. The APB channel touches single 32-bit words and operates in the tens-of-nanoseconds-per-write regime; the AXI4 channel moves 4096-byte tile banks in single-digit microseconds. Keeping these roles strictly separated — never trying to stream bulk data through APB — is the discipline behind the demo’s sub-millisecond FC timings.

Shared DDR as the data exchange. Both the ARM core (through its native AXI bus into the PS DDR controller) and the CGRA’s DMA engine (through an AXI High-Performance slave port on the PS) reach the same physical DDR3. All bulk data exchanged between the two — the test images in `.cnn_sweep`, the FC weight blob in `.cnn_spm`, the inter-layer activations `act400` and `act64`, and the HDMI framebuffer at `0x1F000000` — lives in DDR. The CGRA never reads or writes on-chip memory; the ARM never reads or writes the CGRA’s internal tile memory or scratchpads (those are reachable only as DMA destinations). The DDR is therefore both the message-passing fabric between the two compute domains and the staging area for the HDMI output pipeline that consumes the framebuffer at scan-out time. This “everything goes through DDR” discipline means that the only synchronization primitive the host needs is a properly-ordered `dsb` barrier before kicking the CGRA, ensuring that the activation writes it just performed have actually drained out of the ARM write-buffer into DDR before the CGRA’s DMA tries to read them.

HDMI as a passive consumer. The HDMI output pipeline (`axi_vdma`, `color_convert`, `pixel_unpack`, `rgb2dvi`, `v_tc`) runs continuously in PL and is a passive consumer of the DDR framebuffer. Once the host has configured these IPs at boot through their APB-mapped registers, they require no per-frame ARM intervention — the VDMA simply re-reads the same 921,600-byte region of DDR 60 times per second and pushes the resulting pixel stream out the HDMI TX pins. The application’s role per frame reduces to overwriting the framebuffer’s contents with new pixels through volatile pointer writes; no notification to the HDMI pipeline is required, only the same `dsb` barrier that guarantees the writes have reached DDR before the next VDMA scan begins.

4.2.2 Architectural Partitioning

The software architecture observes a strict separation between **host-side concerns** (executed on the Cortex-A9) and **accelerator-side concerns** (executed on the CGRA fabric). Table 4.1 enumerates the per-stage assignment.

The CGRA is invoked only for the workload pattern in which it dominates the ARM core — dense matrix multiplication with regular weight reuse — leaving all serial,

Table 4.1: Partitioning of software concerns between the ARM core and the CGRA fabric. The “Executor” column gives the hardware target; the implementing source file is named in the “Rationale” column where it adds context.

Concern	Executor	Rationale
Boot, SLCR clock setup, VFP enable, BSS zero	ARM	Required before any application code can run (<code>start.s</code>).
HDMI bring-up: dynclk MMCM, VTC, VDMA, color-convert, pixel-unpack	ARM	Sequential register-write protocol; no acceleration benefit (<code>hdmi_bm.c</code>).
Conv1+ReLU+Pool1+Conv2+ReLU+Pool2	ARM-VFP	The CGRA’s per-pixel Conv kernel saturates its INT32 result FIFO on a $9\text{-MAC} \times 8\text{-channel}$ sum; scalar VFP avoids this and yields the ground-truth float reference (<code>arm_cnn_bm.c</code>).
Flatten + per-image quantization (HWC→CHW, max-abs to INT16)	ARM-VFP	Negligible compute; produces the <code>act400[400]</code> vector consumed by all three FC paths.
FC1 (400×64) and FC2 (64×10) dense matmul	CGRA	The <code>SPM_AUTO_INC</code> + <code>TILE_AUTO_INC</code> + 4-row-parallel pattern is the kernel the CGRA is optimized for (<code>cgra_kernels_cnn.h</code>).
Bias add, ReLU, requantize between FC1 and FC2	ARM	64 scalar operations; faster than configuring a CGRA bypass kernel.
Argmax over 10 FC2 logits	ARM	10-element reduction; trivial.
Cycle-counter timing (PMCCNTR)	ARM	Required for the head-to-head comparison.
Framebuffer rendering: rectangles, glyphs, image blit	ARM	Volatile writes into the HDMI framebuffer at <code>0x1F000000</code> (<code>fb_lib_bm.c</code>).

irregular, or trivially-small operations on the host side. This partitioning is the source of the measured $3.74\times$ speedup on the FC stage and of the demo’s 97% end-to-end classification accuracy.

4.2.3 Per-Frame Execution Flowchart

Where Figure 4.1 answers “what is connected to what,” Figure 4.2 answers “what happens when” — a swim-lane diagram of the actual control flow executed by the demo ELF on every inference frame. The left column captures ARM-side operations; the right column captures CGRA-side operations; dashed green arrows indicate ARM↔CGRA handoffs that cross the boundary introduced in the previous subsection (APB control writes or DDR-resident SG-DMA descriptor chains). The blocks are numbered 1–13 so the step-by-step description that follows the figure can reference each one directly.

The flowchart is hierarchical: the per-frame loop executes indefinitely, while the boot stages run once. The CGRA is invoked *twice* per frame (FC1 and FC2), with the ARM core orchestrating data movement and post-processing in between. The remainder of this subsection walks through every numbered block; Figure 4.3 then zooms into the internal six-step structure of one `cnn_fc_run_group()` call.

Boot phase (steps 1–5, executed once at power-on).

Step 1. Reset vector in `start.s`. On power-on the Cortex-A9 starts execution at the reset vector. The hand-written assembly sets the Vector Base Address Register

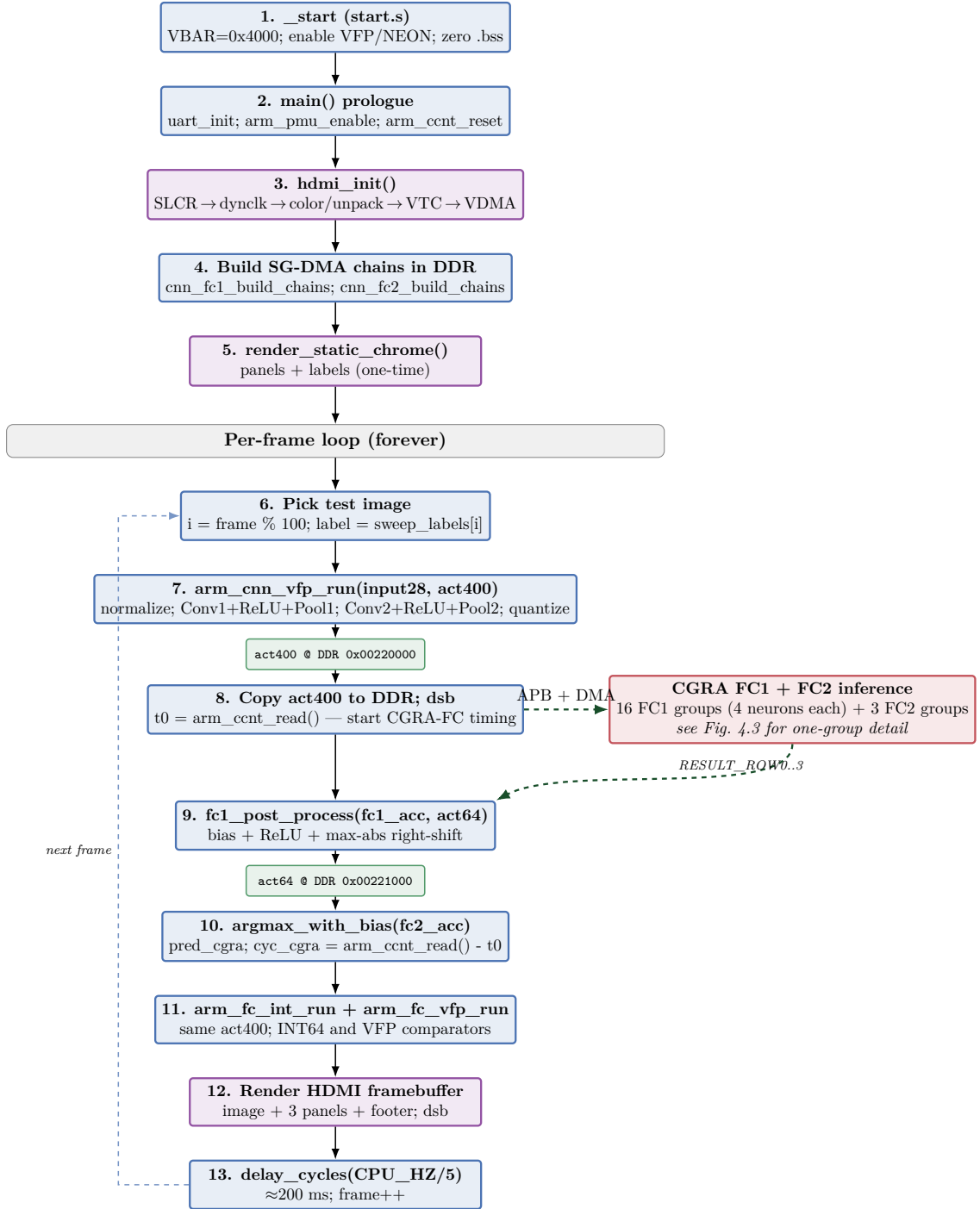


Figure 4.2: Per-frame execution flow. Boot stages (top, above the labelled bar) execute once; per-frame stages execute indefinitely. Blue boxes are ARM-side, red boxes are CGRA-side, purple boxes are HDMI-pipeline operations, and green mini-boxes mark DDR-resident handoff buffers. Dashed green arrows are APB/DMA crossings of the ARM-CGRA boundary. The CGRA-FC block expands to Fig. 4.3.

(VBAR) to `0x00004000`, enables the VFP/NEON coprocessor (CPACR bits `cp10` and `cp11`) and clears `FPEXC`, zeros the `.bss` section in OCM, and finally jumps to the C `main()`. The MMU and the L1 data cache are deliberately left disabled — all subsequent memory accesses are physical and uncached, which keeps the cycle-counter measurements deterministic.

Step 2. `main()` prologue. The first few statements of `main()` bring up the host’s instrumentation infrastructure. `uart_init()` configures the Zynq PS UART0 for 115200 baud, 8N1, on MIO 14/15. `arm_pmu_enable()` writes the Performance Monitor Control Register (PMCR) to enable the on-chip cycle counter, and `arm_ccnt_reset()` clears it so that all subsequent `arm_ccnt_read()` calls return a monotonically-increasing cycle count. Without these calls, the head-to-head FC timings would be impossible.

Step 3. HDMI pipeline bring-up. `hdmi_init()` brings up the five PL IPs that make up the video output. It first unlocks the PS SLCR registers, sets `FCLK0` to 100 MHz (the reference clock that `axi_dynclk` expects), re-locks SLCR, and then calls `hdmi_force_idle()` to quench any leftover state from a prior ELF run. It then programs the MMCM inside `axi_dynclk` to synthesize a 25.175 MHz pixel clock, loads an identity 3×3 colour-matrix into `color_convert`, sets `pixel_unpack` to the `V_24` (3-byte BGR) mode, programs `v_tc` for 640×480 @ 60 Hz timing, and finally arms `axi_vdma` in MM2S mode pointing at the framebuffer at `0x1F000000`.

Step 4. Build SG-DMA descriptor chains in DDR. The host now calls `cnn_fc1_build_chains()` followed by `cnn_fc2_build_chains()`. These two routines populate a region of DDR with linked lists of `cgra_sg_dma_descriptor_t` records — one chain per phase of an FC group (SPM-preload, `ACC_CLR`, MAC-restore, east-readout, plus a one-time east-chain init). After this step, dispatching a 64-neuron FC1 layer or a 10-neuron FC2 layer requires no further descriptor construction at run time; the host simply writes a chain’s head pointer into the `DMA_DESC_HEAD` register and asserts the chain-start bit.

Step 5. Render static chrome. Finally, `render_static_chrome()` paints the panel rectangles, the labels (“CGRA-FC”, “ARM-INT-FC”, “ARM-VFP-FC”), and the screen titles to the HDMI framebuffer. These elements never change across frames, so they are written once at boot rather than on each per-frame render pass. A single `dsb` guarantees the writes have reached DDR before the VDMA’s next scan-out cycle picks them up.

Per-frame phase (steps 6–13, repeated forever).

Step 6. Pick the next test image. The host computes `i = frame % SWEEP_N_IMAGES` to index into the 100-image fixture and looks up `label = sweep_labels[i]` for accuracy bookkeeping. Both arrays live in the read-only `.cnn_sweep` DDR section, so no copy is needed — the ARM operates on pointers directly into the fixture.

Step 7. ARM-VFP convolutional front-end. The function `arm_cnn_vfp_run()` takes the raw `sweep_input28[i]` array as input and fills the output buffer `act400[400]` after running the full floating-point convolutional pipeline on the ARM core. Inside it, `normalize_input()` converts each `uint8_t` pixel into a VFPv3 float and applies

the MNIST whitening transform; Conv1 (3×3 , $1 \rightarrow 8$ channels) is computed in place; `relu_inplace()` clamps negatives to zero; `maxpool_2x2()` reduces to $13 \times 13 \times 8$; the same Conv+ReLU+Pool sequence is repeated for Conv2 (3×3 , $8 \rightarrow 16$); the $5 \times 5 \times 16$ tensor is transposed from HWC to CHW order to match PyTorch’s flatten convention; and finally `quantize_to_int32()` computes the per-image max-abs and rescales the 400 values to the INT16 dynamic range required by FC1. The output, `act400[400]`, is a stack-resident INT32 buffer.

Step 8. Stage the activation in DDR and start timing. The host copies `act400[400]` into the DDR-resident staging buffer at `ACT400_DDR=0x00220000` using a simple C loop of volatile INT32 writes. A `dsb` memory barrier flushes the ARM write-buffer; without it, the CGRA’s DMA engine could begin reading DDR before the ARM’s writes have actually landed. Then `t0=arm_ccnt_read()` captures the starting cycle so the CGRA-FC stage can be timed independently of the Conv stage above.

Step 9. CGRA executes FC1 and FC2. This is the principal ARM $\leftrightarrow$ CGRA hand-off. The host first calls `cnn_fc1_tile_preload(ACT400_DDR)` which broadcasts the 1600-byte `act400` into all four CGRA tile banks via a single 1-D DMA transfer. It then iterates 16 times, once per output-neuron group of four, calling `cnn_fc1_run_group(g, &fc1_acc[g*4])`. Each call triggers the six-step pre-built SG-DMA + CU sequence detailed in Figure 4.3 (SPM preload, ACC_CLR, MAC restore, MAC CU run, east readout, APB drain) and produces four INT32 dot products. The 64-element `fc1_acc[]` array is then post-processed (step 9), restaged in DDR, and the FC2 layer is dispatched in the same way — three groups instead of sixteen, with the trailing two slots discarded since FC2 has only 10 outputs.

Step 10. FC1 post-processing on the ARM. The 64 FC1 accumulator values are passed to `fc1_post_process()`, which adds the per-neuron bias from `mnist_fc1_bias_q[]`, applies ReLU, finds the maximum absolute value across all 64 outputs, and right-shifts the entire buffer so that every value fits in INT16 range. The shifted vector `act64[64]` is then staged in DDR at `ACT64_DDR=0x00221000`, ready to feed FC2. This step is too small (64 elements) to benefit from CGRA acceleration; the ARM finishes it in well under one microsecond.

Step 11. Argmax and FC timing close. After FC2 returns, the host calls `argmax_with_bias(fc2_acc)` which adds the per-class bias from `mnist_fc2_bias_q[]` to each of the ten logits and returns the index of the maximum — the predicted digit class. `t1=arm_ccnt_read()` stops the timing window so `cyc_cgra=t1-t0` records exactly how many cycles the FC stage consumed on the CGRA path.

Step 12. Re-time the same input on the two ARM-only comparators. The host now repeats the FC stage twice more, this time without the CGRA: `arm_fc_int_run(act400, cnn_spm_start)` executes the same INT64-accumulator math entirely on the Cortex-A9 (matching the CGRA’s integer pipeline byte-for-byte), and `arm_fc_vfp_run(act400, cnn_spm_start)` executes the same FC stage with VFPv3 floating-point MACs (the “modern compiler default” comparator). Each run is wrapped in its own `arm_ccnt_read()` window so the three timings are mutually independent and directly comparable. All three paths see the same input `act400`, so the only variable being measured is the FC implementation itself.

Step 13. Render the result to HDMI. `fbm_draw_image28()` blits the raw 28×28 input digit at $8 \times$ scale into a 224×224 region of the framebuffer at the top of the screen. `render_panel()` then redraws each of the three result panels with the latest `PRED`, `GT`, `$\mu s$` , and `FPS` values; `render_footer()` redraws the speedup ratios and the three per-path accuracy counters. All of these primitives are simple volatile writes into the DDR framebuffer; a final `dsb` ensures the writes have drained before the VDMA's next scan-out pass.

Step 14. Inter-frame delay and loopback. `delay_cycles(CPU_HZ/5)` busy-waits for approximately 200 ms by polling the same PMU cycle counter used for the FC timing. This dwell rate gives a defense audience roughly five frames per second of visible inference, which is slow enough to read each prediction comfortably and fast enough to convey live operation. `frame++` and control returns to step 6 with the next image.

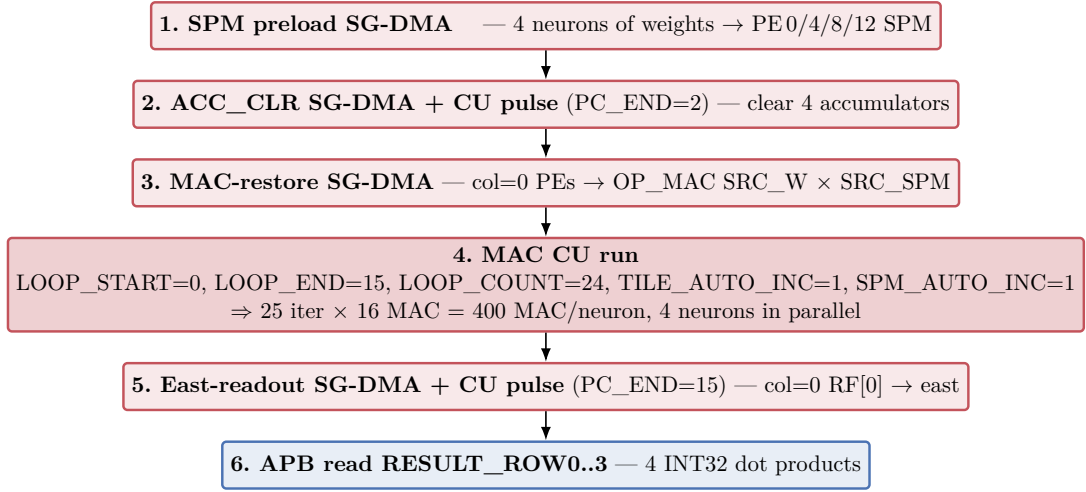


Figure 4.3: The six-step structure of one `cnn_fc_run_group(g)` invocation. Steps 1–5 execute on the CGRA fabric without ARM intervention; step 6 is the only APB read in the per-group critical path. FC1 calls this 16 times per frame (64 outputs); FC2 calls it 3 times (10 outputs; last group has 2 valid neurons).

4.3 Software Implementation

4.3.1 Initialization of Memory Regions

In a Linux deployment the same address map would be accessed through `mmap()` on `/dev/mem` or a UIO device node (`open("/dev/uio0", O_RDWR)` followed by `mmap(..., PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0)`), with the kernel translating between virtual and physical addresses. In the present bare-metal deployment, however, the Cortex-A9 boots with the MMU disabled and the L1 data cache off; all accesses are physical and identity-mapped. Memory-mapped peripherals are accessed by casting their physical addresses to `volatile uint32_t *` and dereferencing them directly. A single header, `cgra.h`, centralizes the address constants and provides inline wrappers for register access (Listing 4.1).

```

1 #define CGRA_BASE          0x43C80000UL  /* APB base, PYNQ-base
   bolt-on */

```

```

2  #define CGRA_PFX_TILE          0x10000000UL  /* tile memory window
    */
3  #define CGRA_PFX_CONFIG        0x20000000UL  /* per-PE config RAM
    window */
4  #define CGRA_PFX_SPM           0x40000000UL  /* per-PE scratchpad
    window */
5  #define CGRA_TILE_BCAST        (CGRA_PFX_TILE | 0x80000000UL)
6
7  static inline void cgra_wr(uint32_t off, uint32_t v) {
8      *(volatile uint32_t *) (CGRA_BASE + off) = v;
9  }
10 static inline uint32_t cgra_rd(uint32_t off) {
11     return *(volatile uint32_t *) (CGRA_BASE + off);
12 }

```

Listing 4.1: Base addresses and register helpers for the CGRA APB and DMA windows (excerpt from `cgra.h`).

The CGRA’s 30 APB control-status registers occupy offsets 0x00–0x84 from `CGRA_BASE`; the application accesses them exclusively through `cgra_wr()`/`cgra_rd()`. Three additional address windows — `CGRA_PFX_TILE`, `CGRA_PFX_CONFIG`, and `CGRA_PFX_SPM` — are *not* directly addressable from the ARM core. They are reachable only as *destinations* of the CGRA’s integrated DMA engine; writing the SPM, for instance, is accomplished by configuring the DMA with `DMA_DST = CGRA_SPM_PE_ADDR(pe, 0)` and triggering a transfer. This routing keeps the high-bandwidth weight and activation traffic on the dedicated DMA path rather than serializing it through the narrow APB bus.

The DDR layout is controlled by the linker script `linker_cnn.ld`. Three custom section attributes redirect large objects to specific DDR regions so that on-chip memory (OCM, 120 KB) is reserved for code, stack, and small `.bss` arrays:

- `.cnn_spm` — 105 KB FC weight blob at DDR 0x00200000.
- `.cnn_sweep` — 78 KB test-image fixture at DDR 0x00240000.
- `.cnn_runtime` — 105 KB float-weight cache (VFP comparator), NOLOAD, at DDR 0x00253CA0.

Each large array in the source carries `__attribute__((section(...)))` to be placed in the appropriate region. The HDMI framebuffer is fixed at 0x1F000000 (640 × 480 × 3 bytes = 921,600 bytes) by the VDMA configuration; the ARM treats it as a flat array of BGR triplets.

4.3.2 Initialization of Weights and CGRA Configuration

The CNN’s weight tensors are produced offline by the Python quantization pipeline (`07_sw/cnn_eval/quantize_cgra.py`) and exported as two binary blobs: `mnist_weights_conv.bin` (2.5 KB, two Conv layers and their biases in INT16/INT32) and `mnist_weights_spm.bin` (105 KB, FC1 and FC2 weights in INT16-in-INT32 SPM layout). These are embedded into the ELF at compile time using the assembler’s `.incbin` directive (Listing 4.2).

```

1  .section .cnn_spm, "a"
2  .global cnn_spm_start
3  cnn_spm_start:

```

```

4      .incbin "../cnn_eval/mnist_weights_spm.bin"
5
6      .section .cnn_conv_spm, "a"
7      .global cnn_conv_start
8  cnn_conv_start:
9      .incbin "../cnn_eval/mnist_weights_conv.bin"

```

Listing 4.2: Weight-blob embedding via the `.incbin` directive. The linker places these sections at fixed DDR addresses.

At runtime, the host code references the blobs via the linker-emitted symbols `cnn_spm_start` and `cnn_conv_start`; no file I/O is required.

Configuration of the CGRA fabric proceeds at two levels. The *microarchitectural level* programs each PE's 16-slot context-memory with a kernel-specific 64-bit instruction word (encoding opcode, two source selectors, destination, route mask, and a 16-bit immediate). The *macroarchitectural level* programs the CU's loop-count registers and auto-increment enables that govern how those PE contexts are sequenced.

For PE configuration, the CGRA exposes a **Configuration Broadcaster**: writing a 64-bit instruction word to the slot-0 address of any PE causes an on-chip FSM to replay the same word into slots 1–15 of that PE within 15 cycles. To install a heterogeneous multi-slot kernel, the application writes slot 0 first (broadcasting an idle value), then overwrites the specific slots that require different instructions. For a 16-PE broadcast (every PE programmed identically), bit 14 of the destination address (CGRA_CFG_BCAST_FLAG) enables a second broadcast plane that fans the write to all 16 PEs in parallel. The inline helper `cgra_config_pe_slot()` assembles the 64-bit word and issues the double-pumped (hi32/lo32) APB-DMA pair that latches it into the PE config RAM.

For the FC kernel the application takes advantage of a higher-level abstraction: `cnn_fc1_build_chains()` and `cnn_fc2_build_chains()` are called once at boot and pre-compute a set of **Scatter-Gather DMA descriptors** in DDR, each describing one phase of the FC group sequence:

- **SPM preload chain** — loads four neurons' worth of weights into the col-0 PEs' scratchpads (per-group);
- **ACC_CLR chain** — broadcasts the accumulator-clear opcode to all four col-0 PEs;
- **MAC-restore chain** — broadcasts the MAC opcode to the same col-0 PEs;
- **East-readout chain** — reprograms col-0 PEs to PASS0 SRC_RFROUTE_E for the result drain;
- **East-chain init** (one-time) — programs col-1/2/3 PEs to PASS0 SRC_WROUTE_E, forming the east-bound result conveyor.

Each chain is a linked list of `cgra_sg_dma_descriptor_t` records:

```

1  typedef struct {
2      uint32_t src;           /* DDR source address */
3      uint32_t dst;          /* CGRA destination (config/SPM/tile) */
4      uint32_t size;         /* byte count */
5      uint32_t next_ptr;     /* DDR pointer to next descriptor, */

```

```

6                                     /* or 0 to terminate the chain */
7 } cgra_sg_dma_descriptor_t;

```

Listing 4.3: SG-DMA descriptor layout used by the FC chain infrastructure (from `cgra.h`).

At runtime, dispatching a full FC group requires no further configuration of the PE fabric: the application writes the head pointer of the appropriate chain to `DMA_DESC_HEAD` and asserts `DMA_CTRL` bit 1; the chain controller fetches each descriptor, executes the transfer, and follows `next_ptr` until termination. This pre-built-chain pattern reduces the per-frame APB write count from several hundred to fewer than fifty, and is the dominant reason the CGRA-FC stage completes in 1.50 ms on silicon.

4.3.3 Reading Input Data

For the MNIST demo the input dataset is embedded into the ELF, eliminating any runtime I/O dependency. The Python utility `07_sw/cnn_eval/emit_sweep_fixture.py` selects 100 representative test images from the MNIST test set, exports their 28×28 grayscale pixel arrays as a single C header file `mnist_sweep_fixture.h`, and tags the arrays with the `SWEEP_RODATA` section attribute (Listing 4.4).

```

1 #define SWEEP_RODATA __attribute__((section(".cnn_sweep")))
2 static const uint8_t SWEEP_RODATA sweep_input28[100][784] = { ... };
3 static const uint8_t SWEEP_RODATA sweep_labels[100] = { ... };

```

Listing 4.4: Test fixture, auto-generated. Linker places the 78 KB array at DDR 0x00240000.

At each frame the application indexes into these arrays with `i = frame % 100`, obtaining `sweep_input28[i]` (the raw `uint8_t[784]` pixel buffer) and `sweep_labels[i]` (the ground-truth class for accuracy bookkeeping). Because the data resides in DDR rather than OCM, no copy is required to make it accessible to the ARM core; the `arm_cnn_vfp_run()` entry point receives a `const uint8_t *` pointer directly into the read-only blob.

The activation buffers consumed by the FC stage are allocated at fixed DDR addresses by the application, not by the linker, so that their physical addresses can be programmed directly into the CGRA DMA’s `DMA_SRC` register without requiring an inverse virtual-to-physical lookup. Two such buffers suffice:

```

1 #define ACT400_DDR 0x00220000UL /* int32[400] act400 between Conv
   and FC1 */
2 #define ACT64_DDR 0x00221000UL /* int32[64] act64 between FC1
   and FC2 */

```

Listing 4.5: Fixed DDR staging addresses for inter-layer activations (from the demo source).

These addresses sit in the gap between the `.cnn_spm` section (which ends at 0x00219A00) and the `.cnn_sweep` section (which begins at 0x00240000), leaving them safely outside any linker-managed region.

4.3.4 Preprocessing of Input Data

The pixel values in `sweep_input28[i]` are unsigned 8-bit integers in the range $[0, 255]$; the trained CNN expects normalized floating-point inputs in the form $(v/255 - \mu)/\sigma$

with $\mu = 0.1307$ and $\sigma = 0.3081$ (the standard MNIST whitening parameters). The ARM-side preprocessing routine, implemented in `arm_cnn_bm.c::normalize_input()`, applies the transformation in a tight VFP loop (Listing 4.6).

```

1 static void normalize_input(const uint8_t *src, float *dst)
2 {
3     const float inv255 = 1.0f / 255.0f;
4     const float mean    = 0.1307f;
5     const float stdinv  = 1.0f / 0.3081f;
6     for (int i = 0; i < 784; ++i)
7         dst[i] = ((float)src[i] * inv255 - mean) * stdinv;
8 }

```

Listing 4.6: Pixel normalization with VFPv3 hard-float.

The output `dst[]` is a 32-bit float array of length 784, populated by VFPv3 single-cycle MAC instructions emitted by GCC under the `-mfpv3 -mfloat-abi=hard` flag combination. This vector then feeds the two convolutional layers, each followed by a ReLU activation and a 2×2 max-pool, yielding successively the $26 \times 26 \times 8$, $13 \times 13 \times 8$, $11 \times 11 \times 16$, and $5 \times 5 \times 16$ feature tensors.

The output of the convolutional front-end is in floating point and in HWC memory order. To match the input format expected by FC1 — a single 400-element integer activation vector in INT16 dynamic range and in CHW order, mirroring the PyTorch tensor-flatten convention against which the FC weights were trained — two further preprocessing steps are required. First, a **transpose** rearranges the $5 \times 5 \times 16$ tensor from HWC to CHW order. Second, a **per-image symmetric quantization** computes `max_abs = max(|x|)` over the 400 elements and rescales each element to fit into INT16 (Listing 4.7).

```

1 float inv_scale = 32767.0f / max_abs;
2 for (int i = 0; i < 400; ++i) {
3     float q = buf_pool2_chw[i] * inv_scale;
4     int32_t qi = (int32_t)(q >= 0.0f ? q + 0.5f : q - 0.5f);
5     if (qi > 32767) qi = 32767;
6     if (qi < -32768) qi = -32768;
7     act400_out[i] = qi;
8 }

```

Listing 4.7: Per-image quantization producing the FC1 input vector `act400`.

The resulting `act400[]` array is composed of 32-bit signed integers whose values lie in $[-32,768, +32,767]$ — i.e., they are INT16-valued but stored in INT32 containers to match the CGRA tile-memory data width. This per-image scaling means the absolute magnitude of the activations is data-dependent, but their dynamic range is fixed; this property is critical for keeping the downstream $\text{INT16} \times \text{INT32}$ FC MAC accumulators inside the 40-bit CGRA saturation envelope.

4.3.5 Hardware Control: APB and DMA Subsystems

After preprocessing, the host transfers `act400[]` into DDR at the fixed staging address `ACT400_DDR`, issues an explicit Data Synchronization Barrier (`asm volatile("dsb" ::: "memory")`) to ensure the writes have drained from the write-buffer into DDR, and then begins the CGRA invocation sequence.

The CGRA exposes its full operating state through the APB CSR file. Table 4.2 summarizes the registers exercised by the FC inference loop; the full 30-register map appears in Chapter 3.

Table 4.2: CGRA APB registers exercised by the MNIST FC inference loop. Offsets are from `CGRA_BASE = 0x43C80000`.

Offset	Mnemonic	R/W	Function in the FC loop
0x00	DMA_CTRL	W	[0]=1: start 1-D DMA; [1]=1: start SG-DMA chain.
0x04	DMA_STATUS	R	[0]=1: busy; [1]=1: done.
0x08	DMA_SRC	W	Source physical address.
0x0C	DMA_DST	W	Destination address (window-encoded).
0x10	DMA_SIZE	W	Byte count for 1-D transfer.
0x14	DMA_SRC_STRIDE	W	Cleared to 0 for 1-D transfers.
0x18	DMA_ROWS	W	Cleared to 0 for 1-D transfers.
0x1C	DMA_COLS	W	Cleared to 0 for 1-D transfers.
0x20	CU_CTRL	W	[0]=1: start CU; [1]=1: soft-reset CU.
0x24	CU_STATUS	R	[0]=1: busy; [1]=1: done.
0x3C	CU_PC_END	W	Last slot the CU will execute before signalling done.
0x40	RESULT_DATA	R	Result FIFO front (PE(3,3) east port). <i>Unused by the FC demo.</i> See note below.
0x48	LOOP_START	W	First slot of the hardware loop body.
0x4C	LOOP_END	W	Last slot of the hardware loop body.
0x50	LOOP_COUNT	W	Loop iteration count (inclusive).
0x58	RESULT_ROW0	R	Latched east-edge output of row 0 (PE 0–3).
0x5C	RESULT_ROW1	R	Latched east-edge output of row 1 (PE 4–7).
0x60	RESULT_ROW2	R	Latched east-edge output of row 2 (PE 8–11).
0x64	RESULT_ROW3	R	Latched east-edge output of row 3 (PE 12–15).
0x74	TILE_BANK_SEL	W	Active tile bank (0–3).
0x78	TILE_AUTO_INC	W	[0]=1: tile read addr advances by 16 each loop iter.
0x7C	DMA_DESC_HEAD	W	SG-DMA chain head pointer (DDR-resident).
0x84	SPM_AUTO_INC	W	[0]=1: SPM read offset advances by 16 each loop iter.

Note on RESULT_DATA: this FIFO is fed by the east port of PE(3,3) every cycle and is the readout channel of the deprecated per-pixel `cnn_conv3x3_pixel()` Conv kernel. The FC demo reads `RESULT_ROW0..3` exclusively; `RESULT_DATA` is documented here for completeness but is not exercised by the locked demo.

A single 1-D DMA transfer is initiated by writing the source, destination, the three zeroed 2-D-mode registers, the byte count, and finally asserting `DMA_CTRL` bit 0. The application then polls `DMA_STATUS` bit 1 until it sets (Listing 4.8). The explicit zeroing of the strided-mode registers is required because the same register file serves both 1-D and 2-D transfers; failing to clear them would re-issue the previous transfer’s 2-D shape on top of the new source/destination, producing partial writes.

```

1 static inline void cgra_dma_start(uint32_t src, uint32_t dst,
   uint32_t bytes)
2 {
3     cgra_wr(CGRA_DMA_SRC,      src);
4     cgra_wr(CGRA_DMA_DST,      dst);
5     cgra_wr(CGRA_DMA_SRC_STRIDE, 0u);    /* disarm 2-D mode */
6     cgra_wr(CGRA_DMA_ROWS,      0u);

```

```

7     cgra_wr(CGRA_DMA_COLS,      0u);
8     cgra_wr(CGRA_DMA_SIZE,      bytes);
9     cgra_wr(CGRA_DMA_CTRL,      1u);    /* trigger */
10  }
11
12  static inline int cgra_dma(uint32_t src, uint32_t dst, uint32_t
    bytes)
13  {
14      cgra_dma_start(src, dst, bytes);
15      return cgra_dma_wait(1000000u);    /* poll DMA_STATUS bit 1 */
16  }

```

Listing 4.8: 1-D DMA helpers used by the FC tile-preload step (excerpt from `cgra.h`).

The DMA destination address encodes the target memory region. Bits [31:28] select between DDR (0x0), tile memory (0x1), PE configuration RAM (0x2), and per-PE scratchpad (0x4). For tile broadcast — writing the same 16 words to all four tile banks in a single transfer — bit 31 of the destination is set, producing the `CGRA_TILE_BCAST = 0x90000000` alias. The full activation preload at the start of each FC inference is therefore the single call `cgra_dma(ACT400_DDR, CGRA_TILE_BCAST, 1600)`.

The CU’s hardware loop is the engine of the dense-matmul pattern. With `LOOP_START = 0`, `LOOP_END = 15`, `LOOP_COUNT = 24`, `TILE_AUTO_INC = 1`, and `SPM_AUTO_INC = 1`, the CU executes 25 iterations of the 16-slot PE program; each iteration advances the tile read address by 16 words and the SPM read address by 16 words. The accumulator in each of the four col-0 PEs (PE 0, 4, 8, 12) integrates $25 \times 16 = 400$ weight-activation products — exactly the dot product required for one FC1 neuron. Four PEs execute four neurons in parallel; one group therefore produces four FC1 outputs per CU invocation, with no host involvement during the 25-iteration MAC loop.

Polling is preferred over interrupt-driven completion for two reasons. First, the per-group CU completion time is approximately 60,000 cycles ($\approx 90 \mu\text{s}$ at 666 MHz), and the entire FC1+FC2 stage — sixteen FC1 groups, three FC2 groups, plus inter-group SG-DMA orchestration — completes in roughly 1.00 million cycles (1.50 ms), well below any meaningful timer-interrupt period. Polling adds no measurable latency on this workload. Second, polling is deterministic in cycle count, whereas the Cortex-A9 interrupt entry cost varies with the current pipeline state. For a thesis whose central performance claim is a sub-millisecond cycle-accurate measurement, deterministic timing of the wait loops is preferable to a marginally lower mean latency.

4.3.6 Output Layers and HDMI Display

After the second FC group sequence terminates, the application reads ten 32-bit signed integers from `RESULT_ROW0–RESULT_ROW3` across three CU invocations (with the final two slots discarded). These ten values are the unbiased FC2 logits in the CGRA’s saturated INT32 accumulator scale. The final output layer of the network — bias addition and argmax — is implemented in pure C on the ARM (Listing 4.9). A softmax is not strictly required because argmax is invariant under any monotonically increasing function of the logits; the final classification result is identical to that of a softmax pipeline.

```

1  static int argmax_with_bias(const int32_t fc2_acc[10])
2  {
3      int32_t v[10];

```

```

4     for (int i = 0; i < 10; ++i)
5         v[i] = fc2_acc[i] + mnist_fc2_bias_q[i];
6     int best = 0;
7     for (int i = 1; i < 10; ++i)
8         if (v[i] > v[best]) best = i;
9     return best;
10 }

```

Listing 4.9: Final classification on the ARM (verbatim from the demo source).

The FC1→FC2 boundary does require a non-trivial post-processing step (`fc1_post_process()`): for each of the 64 FC1 accumulator values, a bias from `mnist_fc1_bias_q[]` is added, a ReLU is applied, and the resulting buffer is requantized to INT16 range via a max-abs right-shift so that downstream FC2 MACs remain inside the INT32 envelope (Listing 4.10). This step is computed on the ARM in a single 64-element loop and consumes a negligible fraction of the inference time.

```

1 static void fc1_post_process(const int32_t fc1_acc[64], int32_t
   act64_out[64])
2 {
3     int32_t tmp[64];
4     int32_t max_abs = 0;
5     for (int n = 0; n < 64; ++n) {
6         int64_t s = (int64_t)fc1_acc[n] +
7                     (int64_t)mnist_fc1_bias_q[n];
8         if (s < 0) s = 0;
9         if (s > 0x7FFFFFFFLL) s = 0x7FFFFFFFLL;
10        tmp[n] = (int32_t)s;
11        if (tmp[n] > max_abs) max_abs = tmp[n];
12    }
13    int shift = 0;
14    while ((max_abs >> shift) > 32767 && shift < 31) shift++;
15    for (int n = 0; n < 64; ++n) act64_out[n] = tmp[n] >> shift;
16 }

```

Listing 4.10: FC1→FC2 post-processing (verbatim from the demo source).

The predicted class label, together with the ground-truth label, the cycle-counter measurement, and the running accuracy total, are then rendered to the HDMI framebuffer at `0x1F000000`. The display geometry places a 224×224 upscaled rendering of the input digit at the top of the 640×480 frame, three result panels (CGRA-FC, ARM-INT-FC, ARM-VFP-FC) directly below, and a footer with the live speedup ratios and per-path accuracy counters.

Three primitives from `fb_lib_bm.c` suffice for the rendering: `fbm_draw_image28(x0, y0, img784, scale)` writes a `scale`×`scale` block of identical-luminance pixels for each of the 784 source bytes, producing the upscaled digit at any integer magnification; `fbm_draw_text(x, y, s, fg, scale)` walks a null-terminated ASCII string and rasterizes each character through a compact 5×7 bitmap font; and `hdmr_rect(x, y, w, h, rgb)` clears the panel backgrounds and footer region at the start of each frame.

Because the Cortex-A9 data cache is disabled in this bare-metal configuration, every store to the framebuffer goes directly to DDR and is observable by the AXI-VDMA on its next scan-out cycle without any explicit cache flush. A single `dsb` instruction at the end of the rendering block guarantees the AXI-VDMA reads a consistent frame; the

VDMA’s free-running MM2S engine then drives the pixel-unpack and color-convert IPs at 25.175 MHz to produce the 640×480 @ 60 Hz HDMI signal.

The complete per-frame software cycle — input read, ARM-VFP Conv+Pool, three FC-path invocations, post-processing, and rendering — completes in approximately 35 ms of wall-clock work, of which the three FC paths account for 1.50 ms, 5.60 ms, and 4.36 ms respectively. A 200 ms inter-frame delay implemented with `delay_cycles(CPU_HZ / 5)` reduces the user-visible refresh rate to a comfortable 5 Hz, allowing a defense audience to observe each prediction and the speedup numbers without strobing.

Table 4.3 summarizes the silicon measurements from the locked demo. The CGRA’s 3.74× wall-clock speedup over scalar-INT ARM on identical math (both INT16 weights × INT32 activations into INT64 accumulators) is the central performance claim of this thesis; the 2.91× speedup over the hardware-VFP comparator is the “honest asterisk” that the same speedup holds even against a modern compiler’s default optimization target.

Table 4.3: Silicon-measured performance of the three FC paths on the locked demo, 201 frames sampled. PYNQ-Z2, Cortex-A9 @ 666 MHz, CGRA @ 50 MHz.

Path	Per-image	FPS	Accuracy	Speedup vs CGRA
CGRA-FC	1.50 ms	668	195/201 (97.0%)	— (baseline)
ARM-INT-FC	5.60 ms	178	195/201 (97.0%)	0.27×
ARM-VFP-FC	4.36 ms	230	201/201 (100%)	0.34×

This concludes the per-frame loop; control returns to the “pick test image” stage of Figure 4.2 and the next input image is processed.